

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
df = pd.read_csv('/forestfires.csv')
print(df.head())
print(df.info())
print(df.describe())
```

[Show hidden output](#)

```
df['month'] = df['month'].astype('category').cat.codes
df['day'] = df['day'].astype('category').cat.codes
```

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['month'] = le.fit_transform(df['month'])
df['day'] = le.fit_transform(df['day'])
df['size_category'] = le.fit_transform(df['size_category'])
```

```
X = df.drop('area', axis=1)
y = df['area']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = Sequential()
model.add(Dense(16, activation='relu', input_dim=X_train.shape[1]))
model.add(Dense(8, activation='relu'))
model.add(Dense(1))
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape` to `input`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=16,
```

```
validation_split=0.2  
)
```

[Show hidden output](#)

```
y_pred = model.predict(X_test)
```

4/4 ————— 0s 31ms/step

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))  
r2 = r2_score(y_test, y_pred)
```

```
print("RMSE:", rmse)  
print("R2 Score:", r2)
```

RMSE: 104.08399917959764
R2 Score: 0.0809554201823417

```
plt.plot(history.history['loss'])  
plt.plot(history.history['val_loss'])
```

```
plt.title("Model Loss")  
plt.xlabel("Epoch")  
plt.ylabel("Loss")
```

```
plt.legend(['Train', 'Validation'])
```

```
plt.show()
```

